AGENTIC REVENUE SYSTEM

EXECUTIVE AND TECHNICAL OVERVIEW

Prepared: October 12, 2026

EXECUTIVE SUMMARY

The Agentic Revenue System is an AI-assisted revenue operations platform. It supports the lifecycle from lead intake and qualification through scoring, outreach, reply handling, call intelligence, IO generation, human approval, external delivery, signature workflow, and audit.

The governing principle is: AI proposes; deterministic software validates and governs; humans approve high-impact actions; adapters perform external side effects; audit records what happened.

WHAT IS AN ?AGENTIC? SYSTEM? ? THE CONCEPT

An agentic system can observe a business situation, reason about the next useful step, propose an action, and continue a workflow toward an objective. Unlike a fixed automation rule, it can interpret context and select an appropriate next step.

In this project, agentic does not mean uncontrolled autonomy. Agents reason and propose; deterministic software controls state, policy, validation, approvals, delivery, and audit.

text
Observe ? Understand ? Propose ? Govern ? Approve ? Execute ? Learn

END-TO-END EXECUTION

1. A lead enters through a CRM, import, webhook, or fixture. The current demo uses fixtures/leads.json; the system does not yet autonomously discover leads from the open internet.
2. The lead becomes a durable lead.created event with an idempotency key.
3. The orchestrator creates or updates a deal in SQLite.

4. The segment registry loads the brand or publisher pack.
5. The deterministic scoring skill calculates fit, intent, engagement, total score, tier, explanation, and fired rules.
6. An agent produces a typed proposal such as an outreach draft, reply classification, transcript digest, or IO draft.
7. The policy engine classifies the action as AUTO, REQUIRE_APPROVAL, or DENY.
8. High-impact actions enter the approval queue.
9. A human approves or rejects the action.
10. Approved actions enter the durable outbox.
11. The delivery service selects the provider adapter and performs the external action.
12. The action is marked sent only after provider success; the provider ID and result are audited.

ARCHITECTURE

text
External source ? durable event ? orchestrator ? agent or skill
?
typed proposal
?
policy engine
AUTO / APPROVAL / DENY
?
outbox or audit
?
provider adapter

The project is a governed modular monolith. It avoids unnecessary microservices while keeping domain, orchestration, governance, agents, skills, segments, and adapters separate.

THE BRAIN ? THE REASONING ENGINE

The Brain is the collection of typed agents and the orchestrator. Agents handle judgment-heavy work: research, outreach drafting, reply classification, prompt-injection detection, call analysis, IO drafting, and pipeline analysis.

The orchestrator selects the agent, supplies context, validates its typed result, and passes it to governance. An agent can propose a message or contract field, but it cannot directly send the message or issue the contract.

THE MEMORY ? THE KNOWLEDGE BASE

The Memory has several layers:

? operational memory: SQLite records deals, events, approvals, outbox work, audit, lessons, and flags;
? business memory: deal stage, fields, score, segment, and timestamps;
? learning memory: reusable lessons extracted from calls and replies;
? configuration memory: YAML segment packs and templates;
? replay memory: fixtures and recorded responses for repeatable demos.

Durable memory ensures a process restart does not lose an approval, pending delivery, or audit record.

THE HANDS ? TOOLS AND INTEGRATIONS

The Hands are provider adapters for HubSpot, Resend, Fireflies, Dropbox Sign, OpenRouter, and the Claude Agent SDK. The delivery layer invokes an adapter through a stable boundary; the adapter translates that action into provider-specific authentication and API calls.

This keeps providers replaceable. Resend could later be replaced by another email provider without changing the Outreach Agent or Policy Engine.

THE RULEBOOK ? GUARDRAILS AND GOVERNANCE

The Rulebook is deterministic application code containing policy decisions, stage guards, scoring thresholds, IO validation, approvals, allowlists, side-effect switches, prompt-injection handling, idempotency, and audit requirements.

Examples include:

```
text
Tier-A first touch ? REQUIRE_APPROVAL
Contract action ? REQUIRE_APPROVAL
Missing recipient ? delivery failure
Prompt injection ? suspicious classification
Side effects disabled ? provider call blocked
Duplicate action key ? no duplicate delivery
```

LEAD INTAKE AND CLASSIFICATION

The lead comes into the system from an external source. In the demo, fixture records simulate this source. A future CRM or prospecting connector can emit the same normalized event.

Classification uses deterministic business inputs such as company fit, employee count, locations, buying signals, pricing-page activity, monthly impressions, inventory types, and engagement. The segment pack supplies segment-specific fields and guidance.

Scoring is implemented in src/revenue_system/skills/scoring.py. The orchestrator applies it, and src/revenue_system/domain/models.py stores the resulting Score.

Scoring answers: How qualified or valuable is this lead?

Policy answers: What is the system allowed to do with this lead?

The LLM is not the final scoring authority. Delegating scoring entirely to an LLM would make results less repeatable, harder to explain and test, and more vulnerable to prompt injection. The LLM may interpret unstructured context, but application code calculates the final score and tier.

SEGMENT PACKS

A segment pack is a configuration package for one business segment. Current packs are packs/brand/ and packs/publisher/.

Packs can define qualification fields, messaging guidance, scoring inputs, and templates. The same agents and orchestrator serve both segments. Adding a segment should normally require a new pack rather than copied application logic.

Pack loading and validation are implemented in src/revenue_system/segments/registry.py and schema.py.

DURABLE STATE

The workflow does not depend on in-memory state. SQLite stores deals, events, approvals, outbox records, audit records, lessons, and system flags. This protects the workflow from restarts, crashes, deployments, retries, and duplicate webhooks.

SQLite is appropriate for this demo. PostgreSQL is recommended for production scale, concurrent workers, and high availability.

WHY AGENTS DO NOT DIRECTLY SEND EMAIL

Agents are responsible for interpretation and proposal generation, not business side effects. They do not directly send email, change CRM data, change pricing, or send contracts.

This separation provides human approval, deterministic validation, prompt-injection protection, provider independence, and complete auditability.

text
Agent ? proposes
Policy ? decides
Human ? approves high-impact actions
Outbox ? stores approved work
Adapter ? performs provider side effect
Audit ? records the result

OUTBOX DELIVERY

src/revenue_system/orchestration/delivery.py performs outbox delivery. It is invoked with:

powershell
python -m revenue_system.cli approvals deliver

It reads pending rows, selects the adapter by action type, sends the request, uses an idempotency key, marks the item sent only after success, and audits failures without losing pending work.

Current paths are:

text
OutboxDeliveryService ? MailAdapter ? Resend
OutboxDeliveryService ? DropboxSignAdapter ? Dropbox Sign

AGENTS AND DETERMINISTIC SKILLS

Agents handle outreach, reply triage, call intelligence, deal desk/IO drafting, research boundaries, and pipeline analysis. They return typed Pydantic models.

Deterministic skills handle scoring, policy, state transitions, IO validation, templates, tracking, lessons, and idempotency-sensitive decisions.

TOOLS USED

? Python: application language.
? Pydantic/Pydantic Settings: typed contracts and configuration.
? SQLite: local durable operational store.
? Typer: operational CLI.
? Jinja2: email and document templates.
? OpenRouter: current live LLM provider using a structured Claude model.
? Claude Agent SDK: supported alternative LLM runner.
? Resend: controlled real email delivery.
? Dropbox Sign: e-signature adapter; test mode is used without a paid plan.
? FastAPI/Uvicorn: webhook and service foundation.
? Git/GitHub: version control and collaboration.

SAFETY AND GOVERNANCE

The system includes human approval for Tier-A outreach and commercial actions, prompt-injection detection, typed schemas, deterministic validation, idempotency, recipient allowlisting, explicit live side-effect control, kill switch, provider error handling, and audit records.

RUNTIME MODES

Mode	Purpose
test	fake runners and isolated tests
replay	deterministic offline demo
record	capture live responses for replay
live	configured provider adapters

Modes are selected independently per integration.

CURRENT VERIFICATION

The project has verified 37 passing tests, both segment packs, live OpenRouter execution, real allowlisted Resend delivery, approval and rejection, approved outbox delivery, audit provider IDs, prompt-injection handling, deterministic IO repair, and Dropbox Sign test-mode request creation.

DROPBOX SIGN WITHOUT A PAID PLAN

ESIGN_TEST_MODE=true creates a test request and records its provider ID, but does not send a real signer notification. The included fixtures/demo-agreement.pdf is fictional and intended only for testing.

PRODUCTION RECOMMENDATIONS

Before large-scale operation, migrate SQLite to PostgreSQL, supervise workers, use a secrets manager, configure signed webhooks, add monitoring and rate limits, configure backups and credential rotation, enforce real domain governance, and obtain an approved paid e-signature plan before disabling test mode.

OPERATIONAL COMMANDS

powershell
python -m revenue_system.cli validate-packs
python -m revenue_system.cli demo --mode replay
python -m revenue_system.cli demo --mode live
python -m revenue_system.cli approvals list
python -m revenue_system.cli approvals approve <approval-id>
python -m revenue_system.cli approvals reject <approval-id>
python -m revenue_system.cli approvals deliver
python -m revenue_system.cli audit <deal-id>
python -m revenue_system.cli kill-switch --enabled
python -m pytest -q

OPERATIONAL WORKFLOW ? A REAL-WORLD EXAMPLE

Assume a brand lead arrives from the CRM with strong fit, a pricing-page visit, and an active marketing hiring signal.

1. The CRM event is normalized as lead.created.
2. The orchestrator stores the lead and loads the brand pack.
3. The scoring skill calculates fit, intent, engagement, total score, and Tier A.
4. The Outreach Agent drafts a personalized first-touch email using relevant lessons.
5. The Policy Engine returns REQUIRE_APPROVAL for a Tier-A first touch.
6. The draft is stored in the approval queue; no email is sent.
7. A revenue operator reviews the subject, body, recipient, and evidence.

8. Approval creates an outbox item with a unique delivery key.
9. The delivery worker calls Resend, restricted to the configured recipient allowlist.
10. Resend returns a provider message ID.
11. The outbox row is marked sent and the provider ID is written to the audit log.
12. Reply Triage classifies the response, and Call Intelligence later extracts qualification fields and lessons.
13. Deal Desk maps trusted terms into an IO and validates budget, dates, and line-item arithmetic.
14. A contract action requires approval again before Dropbox Sign test-mode request creation.

This demonstrates the boundary between intelligence and execution: the agent creates useful work, but the system controls when and how that work affects the outside world.

GOVERNANCE, RISK, AND COMPLIANCE

HUMAN OVERSIGHT

High-impact actions are approval-gated before customer contact, commercial changes, or signature requests.

DATA INTEGRITY

Typed schemas, deterministic validation, idempotency keys, and durable transactions reduce malformed data and duplicate actions.

PROMPT-INJECTION RESISTANCE

Email and transcript content is treated as untrusted data. It cannot change system instructions, grant itself tools, bypass approval, or send a contract.

COMMERCIAL PROTECTION

Budget, dates, line items, pricing, and contract actions are validated and approval-gated. The IO repair layer only reconciles fields against trusted upstream data; it cannot invent commercial terms.

ACCESS AND SIDE-EFFECT CONTROL

Live email requires an explicit recipient allowlist and LIVE_SIDE_EFFECTS_ENABLED=true. E-signature has the same explicit gate. Replay and test modes prevent accidental provider calls during development.

AUDITABILITY

The audit log records scoring, policy decisions, approvals, provider IDs, delivery success, and failures. This gives operators a chronological explanation of what happened.

COMPLIANCE POSTURE

The project provides technical controls supporting reviewability and controlled operations. It is not a complete legal compliance program. Production deployment still requires policies for retention, access control, privacy, vendor agreements, incident response, and regulatory obligations.

SUMMARY OF BENEFITS

? Higher productivity: agents draft and summarize work that would otherwise be manual.
? Consistent qualification: deterministic scoring applies the same rules to every lead.
? Segment flexibility: YAML packs support different segments without duplicated code.
? Safer autonomy: low-risk work can be automated while high-impact actions require approval.
? Provider independence: adapters isolate vendor-specific details.
? Operational resilience: durable events and outbox records survive retries and restarts.
? Continuous learning: conversation lessons can improve future outreach.
? Explainability: scores, policies, approvals, provider responses, and failures are auditable.
? Demo reliability: replay mode makes the workflow reproducible without network access.

NEXT STEPS

IMMEDIATE DEMONSTRATION

1. Validate packs.
2. Run the replay demo.

3. Run live OpenRouter with side effects disabled.
4. Review and approve only the allowlisted test action.
5. Run outbox delivery and inspect the audit record.
6. Demonstrate Dropbox Sign test mode without a paid plan.

NEAR-TERM ENGINEERING

1. Add a web dashboard for approvals, audit, and outbox status.
2. Add real CRM sandbox integration tests.
3. Add Fireflies retrieval and signed webhook verification.
4. Add provider retry, rate-limit, and health-check handling.
5. Remove the remaining serializer warning from malformed test fixtures.
6. Add monitoring and operational metrics.

PRODUCTION SCALE

1. Replace SQLite with PostgreSQL.
2. Run durable workers separately from the CLI process.
3. Add a secrets manager and key rotation.
4. Configure signed webhooks and replay protection.
5. Add backups, alerting, metrics, and incident procedures.
6. Establish retention and privacy policies.
7. Obtain an approved paid e-signature plan before real signer notifications are enabled.